Question 11

Select the rows where the animal is a cat and the age is less than 3.

Solution

```
df[(df['animal'] == 'cat') & (df['age'] < 3)]
```

Output

	animal	age	visits	priority
a	cat	2.5	1	yes
f	cat	2.0	3	no

Explanation

- `&` represents logical **AND**.
- Each condition must be enclosed in parentheses.

Question 12

Select rows where age is between 2 and 4 (inclusive).

Solution

```
df[df['age'].between(2, 4)]
```

Alternative

```
df[(df['age'] >= 2) & (df['age'] <= 4)]
```

Output

	animal	age
a	cat	2.5
b	cat	3.0
f	cat	2.0
j	dog	3.0

Explanation

`between()` is cleaner than writing two comparison operators.

Question 13

Change the age in row 'f' to 1.5.

Solution

```
df.loc['f', 'age'] = 1.5
```

Verify

```
df.loc['f']
```

Output

```
animal      cat
age         1.5
visits      3
priority    no
```

Explanation

`loc[row_label, column_name]` updates a single value.

Question 14

Calculate the total number of visits.

Solution

```
df['visits'].sum()
```

Output

```
19
```

Explanation

`sum()` adds every value in the column.

Question 15

Calculate the mean age for each animal.

Solution

```
df.groupby('animal')['age'].mean()
```

Output

```
animal

cat      2.50
dog      5.00
snake    2.50
```

Explanation

`groupby()` divides the DataFrame into groups.

Then `mean()` calculates the average for each group.

Question 16

Append a new row 'k' and then delete it.

Solution

```
df.loc['k'] = ['rabbit', 1.0, 2, 'yes']
```

Display:

```
print(df.tail())
```

Delete:

```
df = df.drop('k')
```

Explanation

`loc` can add new rows when the index doesn't already exist.

`drop()` removes rows.

Question 17

Count the number of each type of animal.

Solution

```
df['animal'].value_counts()
```

Output

```
dog      4
cat      4
snake    2
```

Explanation

`value_counts()` counts unique values.

Very common in exploratory data analysis.

Question 18

Sort by

- age (descending)
- visits (ascending)

Solution

```
df.sort_values(
    by=['age', 'visits'],
    ascending=[False, True]
)
```

Output (Top Rows)

	animal	age	visits
i	dog	7.0	2
e	dog	5.0	2
g	snake	4.5	1
b	cat	3.0	3

Rows with missing ages (`NaN`) appear at the end by default.

Explanation

Multiple columns can be sorted by passing a list to `by` and corresponding Boolean values to `ascending` .

Question 19

Replace

```
yes → True

no → False
```

Solution

```
df['priority'] = df['priority'].map({
    'yes': True,
    'no': False
})
```

Output

priority
True

priority
True
False
True

Explanation

`map()` replaces values according to a dictionary.

Question 20

Replace every occurrence of

```
snake → python
```

Solution

```
df['animal'] = df['animal'].replace(
    'snake',
    'python'
)
```

Output

animal
cat
cat
python
dog

Explanation

`replace()` substitutes one value with another throughout the column.